

Clase 7 · Módulo 2 — Práctica

Tema Integration Testing + Contract Testing con Pact	Hito Hito 3 — US-03 a US-05 + Gherkin US-01..05 en verde
--	--

Nombre/s: Joaquín Gasco y Juan Riccetto Equipo: 1

Introducción

En este módulo van a escribir integration tests reales contra la API de TaskFlow usando Supertest con una base de datos PostgreSQL de test dedicada, y luego crear su primer contrato Pact entre el frontend y el backend.

Al terminar la clase deberían tener:

- Integration tests para US-03 (crear proyecto), US-04 (listar proyectos) y US-05 (crear tarea)
- Todos los escenarios Gherkin de US-01..05 ejecutando y en verde
- Un contrato Pact consumer generado y verificado por el provider
- La matriz de trazabilidad actualizada

Parte 0 — Setup del entorno (10 min)

Antes de codear, verificá que el entorno de tests está configurado correctamente.

0.1 Estructura del repositorio — rutas importantes

El proyecto TaskFlow es un monorepo. Las rutas relevantes para esta práctica son:

```
taskflow/                                ← raíz del monorepo
├── apps/
│   ├── api/                             ← backend Express + Prisma
│   │   ├── package.json                 ← scripts de test van acá
│   │   ├── .env.test                   ← variables de entorno para tests
│   │   ├── src/
│   │   │   ├── prisma/
│   │   │   │   └── schema.prisma        ← schema de Prisma
│   │   │   ├── routes/
│   │   │   └── services/
│   │   └── tests/                       ← tus integration tests van acá
└── pacts/                               ← contratos Pact generados
```

0.2 Variables de entorno — apps/api/.env.test

Verificá que el archivo apps/api/.env.test existe y tiene este contenido (reemplazá TU_USUARIO con tu usuario de PostgreSQL):

```
# apps/api/.env.test
DATABASE_URL="postgresql://TU_USUARIO@localhost:5432/taskflow_test"
JWT_SECRET="test-secret-super-seguro"
NODE_ENV="test"
```

Si no sabés tu usuario de PostgreSQL, ejecutá `psql -l` y buscá el valor en la columna Owner.

0.3 Scripts en apps/api/package.json

Verificá que estos scripts existen en apps/api/package.json. Si no están, agregálos antes de continuar:

```
"scripts": {  
  "test": "dotenv -e .env.test --override -- vitest run",  
  "test:watch": "dotenv -e .env.test --override -- vitest",  
  "db:migrate:test": "dotenv -e .env.test --override -- prisma migrate deploy --  
    schema=src/prisma/schema.prisma"  
}
```

Nota: el flag `--override` es necesario para que `.env.test` pise el `.env` de la raíz del monorepo.

0.4 Aplicar migraciones a la base de test

Con los scripts ya definidos, aplicamos las migraciones a `taskflow_test`. Ejecutar desde `apps/api/`:

```
cd apps/api  
npm run db:migrate:test
```

Deberías ver: `All migrations have been successfully applied.` Si el comando falla, revisá que `.env.test` tiene la URL correcta con tu usuario de PostgreSQL.

Este comando es seguro de correr varias veces — si no hay migraciones nuevas no hace nada.

0.5 Verificación final

Corré los tests existentes desde `apps/api/` para confirmar que todo funciona:

```
cd apps/api  
npm test
```

Deberías ver los tests de autenticación pasando. Si hay errores de conexión, revisá que `DATABASE_URL` en `.env.test` apunte a `taskflow_test` y no a otra base.



Ejercicio 1 — Integration tests para US-03: Crear Proyecto (30 min)

Creá el archivo `apps/api/tests/projects.integration.test.ts`

El código base ya está disponible en el repositorio como stub — completá los tests marcados con `TODO`.

Consideraciones con PostgreSQL

A diferencia de SQLite in-memory, con PostgreSQL los datos persisten entre ejecuciones. Por eso el aislamiento entre tests es crítico:

- `beforeEach` limpia las tablas antes de cada test con `deleteMany()`
- `afterAll` limpia todos los datos y cierra la conexión con `disconnect()`
- El orden de borrado importa: primero `tasks`, luego `projects`, luego `users` (por las foreign keys)

Estructura del archivo

```
// apps/api/tests/projects.integration.test.ts
import { describe, it, expect, beforeEach, afterAll, afterEach } from 'vitest';
import request from 'supertest';
import { createApp } from '../src/app';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();
const app = createApp();

describe('Proyectos API – US-03 y US-04', () => {
  let token: string;
  let userId: string;

  beforeEach(async () => {
    // Registrar un usuario una sola vez para toda la suite
    const res = await request(app)
      .post('/auth/register')
      .send({ email: 'tester@test.com', password: 'Test1234!' });
    token = res.body.token;
    userId = res.body.user.id;
  });

  afterEach(async () => {
    // Limpiar en orden correcto (foreign keys)
    await prisma.task.deleteMany();
    await prisma.project.deleteMany();
  });

  afterAll(async () => {
    await prisma.task.deleteMany();
    await prisma.project.deleteMany();
    await prisma.user.deleteMany();
    await prisma.$disconnect();
  });

  // TODO: completar los tests de abajo
});
```

1.1 — Happy path: crear proyecto exitosamente

 Criterios de aceptación: CA-10 — nombre obligatorio 3-100 chars; CA-11 — devuelve id y nombre

```
it('crea un proyecto y devuelve 201 con id (@US-03)', async () => {
  const res = await request(app)
    .post('/projects')
    .set('Authorization', `Bearer ${token}`)
    .send({ name: 'TaskFlow MVP', description: 'Primer sprint' });

  // TODO: verificar status
  expect(res.status).toBe(/* ??? */);

  // TODO: verificar que el body tiene id y name
  expect(res.body).toHaveProperty(/* ??? */);
  expect(res.body.name).toBe(/* ??? */);

  // TODO: verificar que el ownerId corresponde al usuario logueado
  expect(res.body.ownerId).toBe(/* ??? */);
});
```

1.2 — Error: nombre vacío

 CA-10 — nombre es obligatorio

```
it('rechaza nombre vacío con 400 (@US-03)', async () => {
  const res = await request(app)
    .post('/projects')
    .set('Authorization', `Bearer ${token}`)
    .send({ name: '', description: 'Sin nombre' });
  expect(res.status).toBe(/* ??? */);
  expect(res.body.error).toMatch(/* regex o string */);
});
```

1.3 — Error: sin autenticación

 CA-12 — requiere JWT válido

```
it('rechaza petición sin token con 401 (@US-03)', async () => {
  const res = await request(app)
    .post('/projects')
    .send({ name: 'Proyecto sin auth' });

  expect(res.status).toBe(/* ??? */);
});
```

Ejercicio 2 — Integration tests US-04 (listar) y US-05 (crear tarea) (20 min)

2.1 — US-04: solo mis proyectos

CA-15 — solo proyectos donde el usuario es owner o miembro

Este test requiere crear un segundo usuario y verificar que no ve los proyectos del primero. Recuerden limpiarlo en afterAll. Se agrega al mismo archivo que los tests anteriores dado que está robando el mismo recurso.

```
it('solo devuelve los proyectos del usuario autenticado (@US-04)', async () => {
  // Crear proyecto del primer usuario
  await request(app).post('/api/projects')
    .set('Authorization', `Bearer ${token}`)
    .send({ name: 'Proyecto de tester1' });

  // Crear un segundo usuario
  const res2 = await request(app).post('/api/auth/register')
    .send({ email: 'otro@test.com', password: 'Test1234!' });
  const token2 = res2.body.token;

  // El segundo usuario lista SUS proyectos
  const list = await request(app).get('/api/projects')
    .set('Authorization', `Bearer ${token2}`);

  expect(list.status).toBe(200);
  // TODO: ¿cuántos proyectos debe ver el segundo usuario?
  expect(list.body.projects).toHaveLength(/* ??? */);
});
```

2.2 — US-05: crear tarea dentro de un proyecto

 **CA-18 — tarea válida devuelve 201 + id; CA-20 — prioridad inválida → 400**

Para estos tests deberán crear el archivo `apps/api/tests/tasks.integration.test.ts`.

```
import { describe, it, expect, beforeAll, afterAll, beforeEach } from 'vitest';
import request from 'supertest';
import { createApp } from '../src/app';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();
const app = createApp();

describe('Tareas API — US-05', () => {
  let token: string;
  let projectId: string;

  beforeAll(async () => {
    const res = await request(app)
      .post('/auth/register')
      .send({ email: 'tester-tasks@test.com', password: 'Test1234!' });
    token = res.body.token;
  });

  beforeEach(async () => {
    await prisma.task.deleteMany();
    await prisma.project.deleteMany();

    const res = await request(app)
      .post('/projects')
      .set('Authorization', `Bearer ${token}`)
      .send({ name: 'Proyecto para tareas' });
    projectId = res.body.id;
  });

  afterAll(async () => {
    await prisma.task.deleteMany();
    await prisma.project.deleteMany();
    await prisma.user.deleteMany();
    await prisma.$disconnect();
  });

  it('crea una tarea con prioridad válida (@US-05)', async () => {
    const res = await request(app)
      .post(`/projects/${projectId}/tasks`)
      .set('Authorization', `Bearer ${token}`)
      .send({ title: 'Implementar login', priority: 'HIGH', status: 'TODO' });

    // TODO: completar las assertions
    expect(res.status).toBe(/* ??? */);
    expect(res.body).toHaveProperty('id');
    expect(res.body.priority).toBe(/* ??? */);
  });

  it('rechaza prioridad inválida con 400 (@US-05)', async () => {
    const res = await request(app)
      .post(`/projects/${projectId}/tasks`)
      .set('Authorization', `Bearer ${token}`)
      .send({ title: 'Tarea mala', priority: 'ULTRA', status: 'TODO' });

    expect(res.status).toBe(/* ??? */);
  });
});
```

Ejercicio 3 — Primer contrato Pact: consumer (frontend) (25 min)

Vas a escribir el test de consumer que genera el contrato Pact para la interacción createProject. Instalá la dependencia si no está (**desde apps/web/**):

```
npm install --save-dev @pact-foundation/pact
```

3.1 — Función cliente que se va a testear

Asegurate de que existe (o creá) el archivo apps/web/src/api/projects.ts:

```
// apps/web/src/api/projects.ts
export async function createProject(
  baseUrl: string,
  name: string,
  description: string,
  token: string
): Promise<{ id: string; name: string; ownerId: string }> {
  const res = await fetch(`${baseUrl}/api/projects`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`,
    },
    body: JSON.stringify({ name, description }),
  });
  if (!res.ok) throw new Error(`Error ${res.status}`);
  return res.json();
}
```

Editar el archivo apps/web/package.json: agregando vitest a la sección de scripts y deps:

```
// apps/web/package.json
"scripts": {
  ...
  "test:pact": "vitest run tests/pact"
}
"devDependencies": {
  "@pact-foundation/pact": "^16.3.0",
  ...
  "vitest": "^1.6.0"
}
```

3.2 — Test Pact consumer

Crear o verificar que está presente `apps/web/tests/pact/createProject.consumer.pact.test.ts`:

```
import { describe, it, expect } from 'vitest';
import { PactV4, MatchersV3 } from '@pact-foundation/pact';
import path from 'path';
import { createProject } from '../../src/api/projects';

const provider = new PactV4({
  consumer: 'taskflow-frontend',
  provider: 'taskflow-api',
  // El contrato se guarda en pacts/ en la raíz del monorepo
  dir: path.resolve(__dirname, '../../pacts'),
});

describe('Consumer Pact – createProject', () => {
  it('POST /api/projects devuelve 201 con id, name y ownerId', async () => {
    await provider
      .addInteraction()
      .given('usuario autenticado con token válido')
      .uponReceiving('una petición para crear proyecto TaskFlow MVP')
      .withRequest('POST', '/projects', (builder) => {
        builder.headers({ 'Content-Type': 'application/json' });
        builder.jsonBody({
          name: MatchersV3.string('TaskFlow MVP'),
          description: MatchersV3.string('desc'),
        });
      })
      .willRespondWith(201, (builder) => {
        builder.jsonBody({
          // TODO: ¿qué matchers usarías para cada campo?
          id: /* MatchersV3.??? */,
          name: /* MatchersV3.??? */,
          ownerId: /* MatchersV3.??? */,
        });
      })
      .executeTest(async (mockServer) => {
        const result = await createProject(
          mockServer.url, 'TaskFlow MVP', 'desc', 'token-de-test'
        );
        expect(result.id).toBeDefined();
        expect(/* ??? */).toBe('TaskFlow MVP');
      });
  });
});
```


Matchers disponibles en MatchersV3

Matcher	Uso
MatchersV3.string(ejemplo)	Verifica que el campo es un string (valor exacto no importa)
MatchersV3.uuid()	Verifica que el campo es un UUID válido (v4)
MatchersV3.integer(n)	Verifica que el campo es un número entero
MatchersV3.boolean(b)	Verifica que el campo es un booleano
MatchersV3.like(objeto)	Verifica que tiene la misma estructura (no valores exactos)
MatchersV3.eachLike(elem)	Verifica que es un array con al menos un elemento con esa estructura

¿Por qué usamos MatchersV3.uuid() en lugar de un string exacto como 'abc-123'?

Porque los pacts estan para validar contratos flexibles y no valores fijos, el string rompe el test si se cambia su valor.

Una vez que el test pasa, verificá que se generó el archivo de contrato:

```
ls pacts/  
# Deberías ver:  
# taskflow-frontend-taskflow-api.json
```

Ejercicio 4 — Verificación del contrato: provider (backend) (20 min)

Ahora el backend va a leer el contrato generado por el frontend y verificar que su implementación lo cumple.

Instalá la dependencia en el backend si no está:

```
cd apps/api && npm install --save-dev @pact-foundation/pact
```

4.1 — Helper: generateTestJWT

Creá apps/api/tests/helpers/auth.helper.ts si no existe:

```
// apps/api/tests/helpers/auth.helper.ts
import jwt from 'jsonwebtoken';

export function generateTestJWT(userId: string): string {
  return jwt.sign(
    { userId, email: 'pact@test.com' },
    process.env.JWT_SECRET ?? 'dev-secret-change-in-production',
    { expiresIn: '1h' }
  );
}
```

4.2 — Test de verificación del provider

Creá apps/api/tests/pact/projects.provider.pact.test.ts:

```
import { Verifier }      from '@pact-foundation/pact';
import path              from 'path';
import { createApp }     from '../../src/app';
import { PrismaClient }  from '../../src/lib/prisma';
import { generateTestJWT } from '../../helpers/auth.helper';
import type { AddressInfo } from 'net';
import { beforeAll, afterAll, describe, it } from 'vitest';

const app = createApp();
const prisma = new PrismaClient();

describe('Provider verification - taskflow-api', () => {
  let server: ReturnType<typeof app.listen>;
  let port: number;

  beforeAll(async () => {
    // Seed mínimo: crear el usuario que usará el stateHandler
    await prisma.user.create({
      data: { id: 'pact-user-1', email: 'pact@test.com', passwordHash: 'hash' }
    });
    server = app.listen(0);
    port = (server.address() as AddressInfo).port;
  });

  it('verifica el contrato del consumer taskflow-frontend', async () => {
    await new Verifier({
```

```

    provider:      'taskflow-api',
    providerBaseUrl: `http://localhost:${port}`,
    pactUrls: [
      path.resolve(__dirname, '../../../../../pacts/taskflow-frontend-taskflow-
api.json'),
    ],
    requestFilter: (req, _res, next) => {
      req.headers['authorization'] = `Bearer ${generateTestJWT('pact-user-1')}`;
      next();
    },
    stateHandlers: {
      'usuario autenticado con token válido': async () => {
        return {};
      },
    },
  }).verifyProvider();
});

afterAll(async () => {
  server.close();
  await prisma.task.deleteMany();
  await prisma.project.deleteMany();
  await prisma.user.deleteMany();
  await prisma.$disconnect();
});
});

```

4.3 — Ejecutar la verificación

```

cd apps/api && npx dotenv -e .env.test --override -- vitest run
tests/pact/projects.provider.pact.test.ts

```

```

# Salida esperada:
# Verifying a pact between taskflow-frontend and taskflow-api
#   una petición para crear proyecto TaskFlow MVP
#     has status code 201 (OK)
#     has a matching body (OK)
#
# 1 interaction, 0 failures

```

 ¿Qué pasa si cambiás 'id' por 'projectId' en la respuesta del controller? Ejecutalo y anotá el error:

Falla la verificación del pacto porque el body esperaba un id y en vez de eso obtuvo un projectId



Ejercicio 5 — Gherkin en verde: US-03, US-04, US-05 (15 min)

Verificá que los escenarios Gherkin de las user stories nuevas ejecutan correctamente.

Si hay step definitions pendientes (undefined), implementalos usando los integration tests como referencia.

5.1 — Ejecutar todos los escenarios

```
cd apps/api && npx cucumber-js --format progress
```

```
# Meta del Hito 3:  
# ✓ US-01 – registro y email duplicado  
# ✓ US-02 – login y credenciales inválidas  
# ✓ US-03 – crear proyecto  
# ✓ US-04 – listar mis proyectos  
# ✓ US-05 – crear tarea con prioridad  
#  
# 10 escenarios (10 passed)  
# 0 pending, 0 undefined
```



¿Cuántos escenarios quedaron en pending o undefined? Lista cuáles y por qué:

No quedaron casos en pending o undefined.

Ejercicio 6 — Actualizar la Matriz de Trazabilidad (10 min)

Completá la siguiente tabla marcando qué tests tenés en verde para el Hito 3. Usá ☒ si pasa, ✗ si falla, ⌚ si está pendiente.

US	CA clave	Unit Test	Integration Test	Gherkin	Contrato Pact
US-01	Registro y mail duplicado	☒	☒	☒	-
US-02	Login y credenciales invalidas	☒	☒	☒	-
US-03	Crear proyecto	☒	☒	☒	☒
US-04	Listar mis proyectos	☒	☒	☒	-
US-05	Crear tarea con prioridad	☒	☒	☒	-

Checklist de entrega — Hito 3

Para la entrega haremos un PR que será mergeado **luego** de obtener feedback.

Antes de hacer el PR, revisá cada punto:

US-03 a US-05

- ✓ POST /api/projects con auth JWT → 201
- ✓ GET /api/projects → solo proyectos del usuario
- ✓ POST /api/projects/:id/tasks → 201 con prioridades válidas
- ✓ Al menos 2 tests de error por endpoint

Gherkin

- ✓ npx cucumber-js ejecuta sin errors ni warnings
- ✓ 0 escenarios pending o undefined
- ✓ Todos los escenarios de US-01..05 en verde

Contrato Pact

- ✓ pacts/taskflow-frontend-taskflow-api.json generado
- ✓ Provider verification: 0 failures

Calidad

- ✓ ESLint y Prettier: 0 errores
- ✓ TypeScript compila sin errores (tsc --noEmit)
- ✓ Coverage de líneas ≥ 70%

Trazabilidad

- ✓ Matriz actualizada con US-03, US-04, US-05
- ✓ Tests etiquetados con @US-03 / @US-04 / @US-05
- ✓ Pipeline CI en verde
